

nginx JS — Dynamic Reconfiguration Guide (for config checkpoint/restore)

Table of Contents

1. Overview	1
1.1. Checkpoint scope summary	2
2. Worker-level checkpoint	2
2.1. How it works	2
2.2. Example: manual checkpoint via admin endpoint	2
2.3. Auto-checkpoint on successful apply	3
2.4. Multi-step migration with intermediate checkpoints	4
3. Cross-worker checkpoint (SharedWorker)	4
3.1. Architecture	5
3.2. SharedWorker checkpoint store	5
3.3. Worker helpers: serialise and apply a checkpoint	5
3.4. Example: save and restore via HTTP endpoints	7
4. Persistent checkpoint (survives nginx reload)	8
4.1. The problem	8
4.2. Pattern: write checkpoint to file, read back via js_preprocess	8
4.3. Flow diagram: full persistent checkpoint lifecycle	9
5. Checkpoint-based A/B testing with controlled rollout	9
6. Caveats	10
6.1. NginxSnapshot is not serialisable	10
6.2. Checkpoint staleness	10
6.3. Concurrent checkpoint writers	10
6.4. Checkpoint and nginx reload	10



DRAFT FOR COMMENTS — This document is a work in progress. Content, API names, and examples are subject to change before release.

1. Overview

A **checkpoint** is a named, persistent copy of config state that survives across multiple requests and can be restored on demand.

This is distinct from the *request-scoped* `snapshot()` / `restore()` covered in the *for single worker* guide: a snapshot lives only for the duration of the request that created it; a checkpoint lives until nginx

restarts (or longer, if externalised).

1.1. Checkpoint scope summary

Scope	Storage	Survives request end	Survives nginx reload
Request snapshot	JS local variable in handler	No (GC'd after handler)	No
Worker checkpoint	JS module-level variable	Yes	No (workers are restarted)
Cross-worker checkpoint	SharedWorker memory	Yes	No (SharedWorker restarted)
Persistent checkpoint	External file / env var (via <code>js_preprocess</code>)	Yes	Yes (re-read at next startup)

This guide covers the last three rows.

2. Worker-level checkpoint

2.1. How it works

A module-level variable in `js_source` persists for the lifetime of a worker process. Any request handler in that worker can read or write it.

```
// js_source: worker_checkpoint.js

// The module-level map is shared across all requests in this worker.
const checkpoints = new Map(); // name → NginxSnapshot

function saveCheckpoint(name, snap) {
  checkpoints.set(name, snap);
}

function loadCheckpoint(name) {
  return checkpoints.get(name) || null;
}
```

2.2. Example: manual checkpoint via admin endpoint

```
// POST /admin/checkpoint?name=stable → save current config
// POST /admin/restore?name=stable → restore that checkpoint

const adminLoc = nginx.http.servers[0].locations
```

```

    .find(l => l.path === '/admin/checkpoint');

adminLoc.handler = (req) => {
  const name = req.getVar('arg_name') || 'default';

  if (req.method === 'POST' && req.uri.startsWith('/admin/checkpoint')) {
    const snap = req.location.snapshot();
    saveCheckpoint(name, snap);
    req.json({ saved: name });
  } else if (req.method === 'POST' && req.uri.startsWith('/admin/restore')) {
    const snap = loadCheckpoint(name);
    if (!snap) {
      req.respond(404, {}, 'checkpoint not found: ' + name);
      return;
    }
    snap.restore();
    req.json({ restored: name });
  }
};

```



A checkpoint stored this way is **per-worker**. The other workers still have the old config. To propagate the restore to all workers, combine this with the SharedWorker broadcast strategy from the *for all workers* guide.

2.3. Auto-checkpoint on successful apply

Automatically save a "last-known-good" checkpoint every time a config change is validated and applied:

```

loc.handler = async (req) => {
  const snap = req.location.snapshot();    // save candidate rollback point

  try {
    const cfg = JSON.parse(await req.readBody());
    req.location.root = cfg.root;

    const ok = await probeBackend(req);
    if (!ok) { throw new Error('probe failed'); }

    // Success – promote snapshot to "last-known-good" checkpoint
    const lkg = req.location.snapshot();    // re-snapshot after apply
    saveCheckpoint('last-known-good', lkg);

    req.json({ applied: true });
  } catch (e) {
    snap.restore();
  }
};

```

```
    req.respond(500, {}, 'apply failed: ' + e.message);
  }
};
```

2.4. Multi-step migration with intermediate checkpoints

Some config changes involve multiple fields that must be updated atomically or rolled back step by step.

```
loc.handler = async (req) => {
  const steps = JSON.parse(await req.readBody());

  // Checkpoint before every step so we can roll back to any intermediate state
  const checkpointStack = [];

  try {
    for (const step of steps) {
      checkpointStack.push(req.location.snapshot()); // save this step's start

      applyStep(step, req);

      const ok = await probeStep(step, req);
      if (!ok) { throw new Error('step failed: ' + step.name); }
    }

    req.json({ applied: steps.length });
  } catch (e) {
    // Roll back to the checkpoint taken before the failing step
    const rollback = checkpointStack[checkpointStack.length - 1];
    if (rollback) { rollback.restore(); }

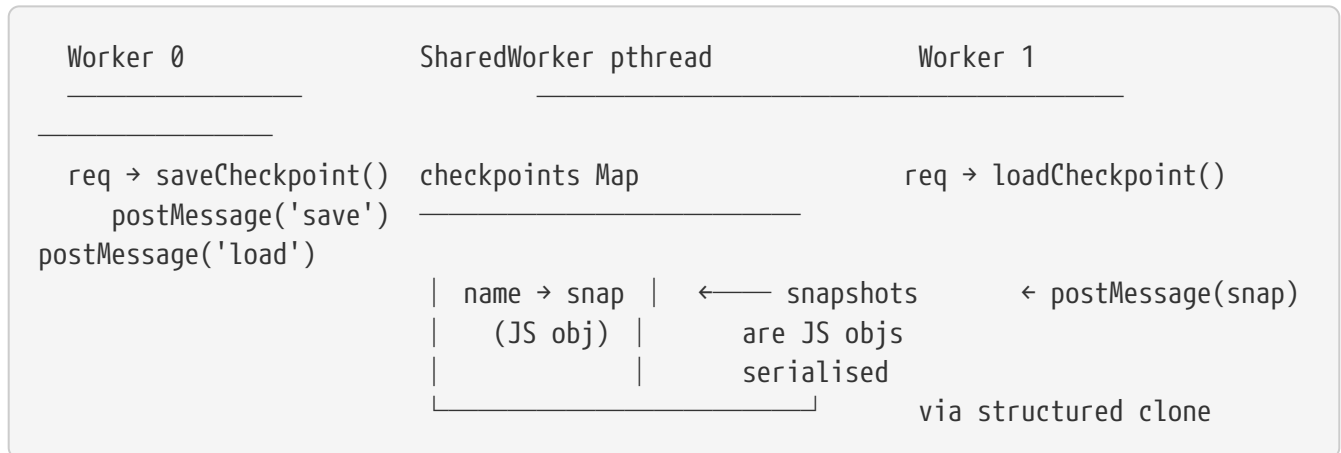
    req.respond(500, {}, e.message);
  }
};

function applyStep(step, req) {
  if (step.type === 'set-root') { req.location.root = step.value; }
  if (step.type === 'set-timeout') { req.location.proxy.readTimeout = step.value; }
  // ...
}
```

3. Cross-worker checkpoint (SharedWorker)

3.1. Architecture

The SharedWorker holds the checkpoint store. Any worker can save or load a checkpoint by posting a message and awaiting the reply.



NginxSnapshot is an opaque C-backed object — it cannot be serialised via structured clone. The SharedWorker checkpoint stores the **raw property values** (plain JS objects), not the **NginxSnapshot** handle itself.

3.2. SharedWorker checkpoint store

```
// cpstore.js (SharedWorker)

const store = new Map(); // name → plain JS object with property values

onmessage = (msg) => {
  const { type, name, data } = msg.data;

  if (type === 'save') {
    store.set(name, data);
    postMessage({ type: 'saved', name });
  } else if (type === 'load') {
    const data = store.get(name);
    postMessage({ type: 'loaded', name, data: data || null });
  } else if (type === 'list') {
    postMessage({ type: 'list', names: [...store.keys()] });
  }
};
```

3.3. Worker helpers: serialise and apply a checkpoint

Because **NginxSnapshot** is opaque, checkpoints are stored as plain objects:

```

// js_source: cross_worker_checkpoint.js

const cpStore = new SharedWorker('cpstore.js');

// Serialise the current location config to a plain object
function captureToPlainObject(req) {
  const loc = req.location;
  return {
    root: loc.root,
    sendfile: loc.sendfile,
    keepaliveTimeout: loc.keepaliveTimeout,
    defaultType: loc.defaultType,
    proxy: loc.proxy ? {
      readTimeout: loc.proxy.readTimeout,
      connectTimeout: loc.proxy.connectTimeout,
      buffering: loc.proxy.buffering,
    } : undefined,
  };
}

// Apply a plain-object checkpoint back to the location
function applyFromPlainObject(req, data) {
  if (!data) { return; }
  if (data.root !== undefined) { req.location.root = data.root; }
  if (data.sendfile !== undefined) { req.location.sendfile = data.sendfile; }
  if (data.keepaliveTimeout !== undefined) { req.location.keepaliveTimeout = data.keepaliveTimeout; }
  if (data.defaultType !== undefined) { req.location.defaultType = data.defaultType; }
  if (data.proxy?.readTimeout !== undefined) { req.location.proxy.readTimeout = data.proxy.readTimeout; }
  if (data.proxy?.connectTimeout !== undefined) { req.location.proxy.connectTimeout = data.proxy.connectTimeout; }
  if (data.proxy?.buffering !== undefined) { req.location.proxy.buffering = data.proxy.buffering; }
}

// Save current location config as a named checkpoint in the SharedWorker
async function saveCheckpoint(req, name) {
  const data = captureToPlainObject(req);
  return new Promise(resolve => {
    cpStore.onmessage = (msg) => {
      if (msg.data.type === 'saved' && msg.data.name === name) { resolve(); }
    };
    cpStore.postMessage({ type: 'save', name, data });
  });
}

```

```
// Load a named checkpoint from the SharedWorker and apply it to this worker
async function loadCheckpoint(req, name) {
  return new Promise((resolve, reject) => {
    cpStore.onmessage = (msg) => {
      if (msg.data.type === 'loaded' && msg.data.name === name) {
        if (!msg.data.data) { reject(new Error('checkpoint not found: ' +
name)); return; }
        applyFromPlainObject(req, msg.data.data);
        resolve();
      }
    };
    cpStore.postMessage({ type: 'load', name });
  });
}
```

3.4. Example: save and restore via HTTP endpoints

```
loc.handler = async (req) => {
  const name = req.getVar('arg_name') || 'default';

  if (req.uri === '/admin/checkpoint/save') {
    await saveCheckpoint(req, name);
    req.json({ saved: name });
  } else if (req.uri === '/admin/checkpoint/restore') {
    try {
      await loadCheckpoint(req, name);
      req.json({ restored: name });
    } catch (e) {
      req.respond(404, {}, e.message);
    }
  } else if (req.uri === '/admin/checkpoint/list') {
    const names = await new Promise(resolve => {
      cpStore.onmessage = (msg) => {
        if (msg.data.type === 'list') { resolve(msg.data.names); }
      };
      cpStore.postMessage({ type: 'list' });
    });
    req.json({ checkpoints: names });
  }
};
```

4. Persistent checkpoint (survives nginx reload)

4.1. The problem

When nginx reloads (`nginx -s reload`) or restarts, all workers and the SharedWorker are replaced. In-memory checkpoints are lost.

To survive a reload, checkpoints must be externalised — written to a file or database before nginx shuts down, and re-read at the next startup.

4.2. Pattern: write checkpoint to file, read back via `js_preprocess`

`js_preprocess` runs during config parse and can read files. A checkpoint can be written to a JSON file by a request handler and read back at next startup.

```
// js_source: persistent_checkpoint.js
// (uses Worker for async file I/O – not shown in detail here)

async function persistCheckpoint(name, data) {
  const path = '/var/run/nginx/checkpoints/' + name + '.json';
  // Write via a Worker that can do synchronous file I/O
  const w = new Worker('filewriter.js');
  w.postMessage({ path, data: JSON.stringify(data) });
  await new Promise(resolve => { w.onmessage = resolve; });
}
```

```
// js_preprocess: read_checkpoint.js
// Runs during config parse; injects config text based on saved checkpoint.

const fs = require('fs');
const path = '/var/run/nginx/checkpoints/startup.json';

try {
  const raw = fs.open(path, 'r').readAsString();
  const data = JSON.parse(raw);

  // Generate nginx config fragment from checkpoint data
  config.write('location /from-checkpoint {\n');
  config.write('    root ' + data.root + ';\n');
  config.write('}\n');
} catch (e) {
  // No checkpoint file – use defaults from nginx.conf
}
```

```
}
```

4.3. Flow diagram: full persistent checkpoint lifecycle

```

Nginx starts / reloads
|
[]
js_preprocess runs
|   reads /var/run/nginx/checkpoints/startup.json
|   injects config text (if file exists)
|
[]
nginx.conf parsed → workers fork
|
|   (nginx is running)
|
[]
Request: POST /admin/checkpoint?name=stable
|   captures current config → plain JS object
|   writes to /var/run/nginx/checkpoints/stable.json (via Worker)
|
Request: POST /admin/restore?name=stable
|   reads from SharedWorker store OR file
|   applies to this worker (write_mode='global')
|   broadcasts to other workers via SharedWorker fan-out
|
nginx -s reload
|
[]
js_preprocess reads stable.json → injects into new config
Workers fork with checkpoint baked in
```

5. Checkpoint-based A/B testing with controlled rollout

Checkpoints can represent named config states (e.g. "control", "experiment-v1", "experiment-v2"). Traffic routing between states is controlled per-request.

```
// js_source: ab_checkpoint.js

const cpStore = new SharedWorker('cpstore.js');

// Pre-load both configs at init time
// (assumes they were previously saved as named checkpoints)

loc.handler = async (req) => {
  // Route by cookie or percentage
  const bucket = req.getVar('cookie_ab_bucket');
```

```

const target = (bucket === 'experiment') ? 'experiment-v1' : 'control';

// Apply per-request (local write_mode – does not affect other requests)
req.location.setWriteMode('local');
req.location.setReadMode('local');

try {
  await loadCheckpoint(req, target); // applies to local snapshot
} catch (e) {
  // Fall back to global (control) config – no local mutation applied
}

req.sendFile('/index.html');
// local snapshot is automatically discarded at request end
};

```

6. Caveats

6.1. NginxSnapshot is not serialisable

`NginxSnapshot` handles are C-backed opaque objects. They cannot be passed to `postMessage` or serialised via `JSON.stringify`. Cross-worker and persistent checkpoints must store plain JS objects (see `captureToPlainObject` above) and apply them field-by-field using setters.

6.2. Checkpoint staleness

A checkpoint captures a point-in-time view. If nginx's upstream peer list changes (peers added or removed), a stale checkpoint may attempt to set weights on non-existent peers. `restore()` (and `applyFromPlainObject`) are best-effort and silently ignore such errors.

6.3. Concurrent checkpoint writers

Module-level variables and `SharedWorker` state are not automatically mutex-protected. If two requests write checkpoints concurrently, the last writer wins. For critical checkpoint writes, use a flag guard similar to `mutationInProgress` from the *for single worker* guide.

6.4. Checkpoint and nginx reload

Files written by `persistCheckpoint` during a request are not guaranteed to be flushed before `nginx -s reload` starts the new worker processes. If the reload is triggered immediately after a checkpoint write, the new `js_preprocess` run may read a partial or previous file. Use atomic rename-from-temp to mitigate:

```

// In filewriter.js Worker:
// 1. Write to /var/run/nginx/checkpoints/.stable.json.tmp

```

```
// 2. Atomically rename to /var/run/nginx/checkpoints/stable.json
```